

## Exercises for Lecture: Practical Parallel Programming

### Project 3 (Simulation)

#### Problem 1 (Physical Simulation)

[25+5 Points]

In this project, we will study the motion of bodies under the influence of (Newtonian) gravity caused by the bodies' masses. In general, the problem cannot be solved analytically; therefore, simulation is of practical relevance.

Let  $n$  bodies  $K_1, \dots, K_n$  with the masses  $m_1, \dots, m_n$  be given. Our simulation is 2-dimensional, i.e., every body is associated with a 2-dimensional position vector  $\vec{x}$  and a 2-dimensional velocity vector  $\vec{v}$ .

The simulation is performed in discrete steps of equal length  $\Delta t$ . In every step  $s$ , the simulation computes from the position and velocity vectors  $\vec{x}_i^{(s)}, \vec{v}_i^{(s)}$  the position and velocity vectors  $\vec{x}_i^{(s+1)}, \vec{v}_i^{(s+1)}$  for the next simulation step  $s + 1$  (see the formulas below).

In `/scratch/ppp2026/3/` you can find data for the initial situation (format description below) and a skeleton for implementation. The user can choose the number of simulation steps  $S$ , the parameter  $\Delta t$  and the initial situation (by a file). The final situation of the simulation is saved to a file in the same format. At the end, the program prints the interaction rate  $n \cdot (n - 1) \cdot \frac{S}{T}$  (where  $T$  is the total time of the simulation), i.e., the average number of interactions per time unit. The total momentum  $\sum_{i=1}^n m_i \vec{v}_i$  should remain approximately constant during the simulation (more precisely, the relative change should be close to 0).

- (a) Implement (in `parallel.c`) the  $n$ -body simulation with OpenMP parallelism. [4 Points]
- (b) Add MPI parallelism to your implementation. [10 Points]
- (c) Use Newton's third law [4 Points]

$$m_i \cdot \vec{a}_{i,j} = -m_j \cdot \vec{a}_{j,i}$$

to reduce the number of local computations in each MPI process. In other words: change the OpenMP part of your implementation to save almost half of the computations that affect "local" bodies only.

- (d) Use Newton's third law globally, i.e., also between MPI processes to save almost half of the computations over all bodies. [6 Points]
- (e) Add the functionality to save images with the current positions of the bodies (as PBM or PGM images) every  $d$  simulation steps (with user-specified  $d$ ). This way, small animations can be created (use, e.g., the command `animate` to play the animations). [1 Point]
- (f) (Only for Master:) [5 Points]

To accelerate the simulation further (trading precision for speed), one can replace bodies  $K_1, \dots, K_l$  that are close together by a surrogate body which has a mass equal to the sum of the masses of the individual bodies  $M = \sum_{i=1}^l m_i$  and is located at the center of gravity

$$\frac{1}{M} \sum_{i=1}^l m_i \vec{x}_i$$

of the individual bodies. When computing the gravitational influence on far away bodies, one uses the surrogate body instead of the individual bodies.

Arrange the MPI processes in a two-dimensional grid with  $p_x$  and  $p_y$  processes in the  $x$ - and  $y$ -dimensions. The main program computes  $p_x$  and  $p_y$  such that  $p_x \cdot p_y$  equals the total number  $p$  of MPI processes and  $p_y \leq p_x$  (cf. members `px` and `py` in the `TaskInput` structure).

To group the bodies in groups, sort the bodies by their  $x$ -coordinates and form  $p_x$  (equally-sized) partitions according to the  $x$ -coordinate. In each partition, sort the bodies by their  $y$ -coordinate and form  $p_y$  sub-partitions. (In total, this yields  $p_x \cdot p_y$  sub-partitions.) Every sub-partition is assigned to an MPI process and each process computes a surrogate body for “its” bodies and communicates the center of mass and the total mass to the other processes.

In the acceleration computations, the usual computations (including Newton’s third law) are performed locally (within each MPI process); globally (between MPI processes), the surrogate bodies are used to compute the influence on each local body. (Newton’s third law is not used non-locally.) Note: due to the approximation, the momentum can differ greatly and the motion of the bodies may become unphysical.

Use `long double` (96 bits) in the computations; the corresponding MPI datatype is `MPI_LONG_DOUBLE`. Do not modify the `id` field in the `body` structures, it is used to write the bodies to the output file in the same order as the input file.

Upload your solution (file `parallel.c` only) in ILIAS. Do not miss the deadline! Late submissions cannot be accepted due to legal regulations! Work on PPP projects is strictly one person (no teams)! Use of AI to produce or analyze code is not allowed!

### Data format:

The initial and final configurations of the system is stored in a file with the following format. In the first row, the number of bodies is given. This is followed by one line per body which consists of 5 numbers: the mass (in  $kg$ ), the  $x$ - and  $y$ -positions (in  $m$ ), the velocity in  $x$ - and  $y$ -directions (in  $\frac{m}{s}$ ). Single-line comments are introduced by `#`. Example:

```
2
1.98e30    0          0  0  0      # sun
5.977e24  1.496e11  0  0  29716   # earth
```

Use this example (in file `sun_earth.dat`) to test the (sequential) implementation. The earth should (almost) go around the sun in about 366 steps of 86400 seconds (= 1 day). Due to the imperfections in the simulation, the orbit increases with every revolution.

Suitable parameters for the simulation (number of steps, duration of one step, image parameters) can be found as comments in the simulation files.

### Mechanics:

According to Newton’s theory of gravity, a body  $K_j$  exercises a gravitational acceleration  $\vec{a}_{i,j}^{(s)}$  on another body  $K_i$  ( $i \neq j$ ) in time step  $s$ :

$$\vec{a}_{i,j}^{(s)} = G \cdot \frac{m_j \cdot (\vec{x}_j^{(s)} - \vec{x}_i^{(s)})}{|\vec{x}_j^{(s)} - \vec{x}_i^{(s)}|^3}$$

The gravitational constant  $G$  has the value  $G \approx 6,674 \cdot 10^{-11} \frac{m^3}{kg \cdot s^2}$ . The total acceleration  $\vec{a}_i^{(s)}$  on body  $K_i$  is:

$$\vec{a}_i^{(s)} = \sum_{\substack{1 \leq j \leq n \\ i \neq j}} \vec{a}_{i,j}^{(s)}$$

The approximations for the new positions  $\vec{x}_i^{(s+1)}$  and the new velocities  $\vec{v}_i^{(s+1)}$  in the next simulation step  $s + 1$  are computed from  $\vec{x}_i^{(s)}$ ,  $\vec{v}_i^{(s)}$ ,  $\vec{a}_i^{(s)}$ ,  $\vec{a}_i^{(s+1)}$  and  $\Delta t$ :

$$\begin{aligned}\vec{x}_i^{(s+1)} &= \vec{x}_i^{(s)} + \vec{v}_i^{(s)} \cdot \Delta t + \frac{1}{2} \cdot \vec{a}_i^{(s)} \cdot (\Delta t)^2 \\ \vec{v}_i^{(s+1)} &= \vec{v}_i^{(s)} + \frac{1}{2}(\vec{a}_i^{(s)} + \vec{a}_i^{(s+1)}) \cdot \Delta t\end{aligned}$$

Since  $\vec{v}_i^{(s+1)}$  depends on both  $\vec{a}_i^{(s)}$  and  $\vec{a}_i^{(s+1)}$ , the values for the next simulation step  $s + 1$  are computed in the following order: first  $\vec{x}_i^{(s+1)}$ , then  $\vec{a}_i^{(s+1)}$  and finally  $\vec{v}_i^{(s+1)}$ .